

Vision Transformer



Vision Transformer(ViT)란?

Vision Transformer(ViT)는 이미지를 처리하기 위해 Transformer 구조를 적용한 딥러닝 모델이다.

기존 CNN(Convolutional Neural Network)이 합성곱(Convolution)으로 특징을 추출했다면, ViT는 이미지를 작은 패치(Patch) 단위로 잘라서 Transformer에 입력한다.

대표 논문은 구글의

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

<https://arxiv.org/abs/2010.11929>

이다.

[A Visual Guide to Vision Transformers:](#)

<https://discuss.pytorch.kr/t/vision-transformer-a-visual-guide-to-vision-transformers/4158>

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

<https://arxiv.org/abs/2010.11929>

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale은 2020년 구글 리서치가 발표한 컴퓨터 비전 논문으로, 흔히 **Vision Transformer(ViT)** 논문으로 알려져 있다. 이 연구는 자연어 처리에 성공적으로 사용된 트랜스포머 모델을 이미지 인식 분야에 도입하며, 대규모 데이터셋에서 CNN을 능가할 수 있음을 입증했다.

핵심 사실

- **저자:** Alexey Dosovitskiy 외 (Google Research, 2020)
- **발표:** 2020년 10월, arXiv 프리프린트
- **모델명:** Vision Transformer (ViT)
- **주요 기여:** 트랜스포머 기반 이미지 분류 최초 성공 사례
- **영향:** CNN 중심의 비전 모델 패러다임에 근본적 전환 제시

왜 등장했는가?

Transformer는 원래 NLP(자연어 처리)에서 사용되었다.

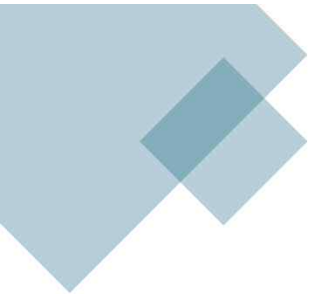
예:

- 문장을 단어 단위로 나눔
- 단어 간 관계를 Attention으로 학습

ViT는 이 아이디어를 이미지에 적용하였다.

즉:

NLP	Vision Transformer
문장(sentence)	이미지(image)
단어(word)	패치(patch)
토큰(token)	패치 임베딩
Self-Attention	이미지 영역 간 관계 학습



핵심 아이디어

이미지를 작은 조각으로 잘라서 "단어처럼" 처리한다.

예를 들어:

- 입력 이미지: $224 \times 224 \times 3$
- 패치 크기: 16×16

그러면:

$$224 / 16 = 14$$

즉,

$$14 \times 14 = 196$$

개의 패치가 생성된다.

각 패치는 하나의 토큰처럼 Transformer에 들어간다.



전체 구조

입력 이미지
↓
Patch 분할
↓
Flatten
↓
Linear Embedding
↓
Positional Encoding 추가
↓
Transformer Encoder
↓
MLP Head
↓
분류 결과

1. Patch 분할

예:

224×224 이미지

를

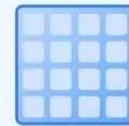
16×16

패치로 자른다.



Patch Embeddings (ViT)

Divide image into fixed-size patches



224×224 image



16×16 Patches

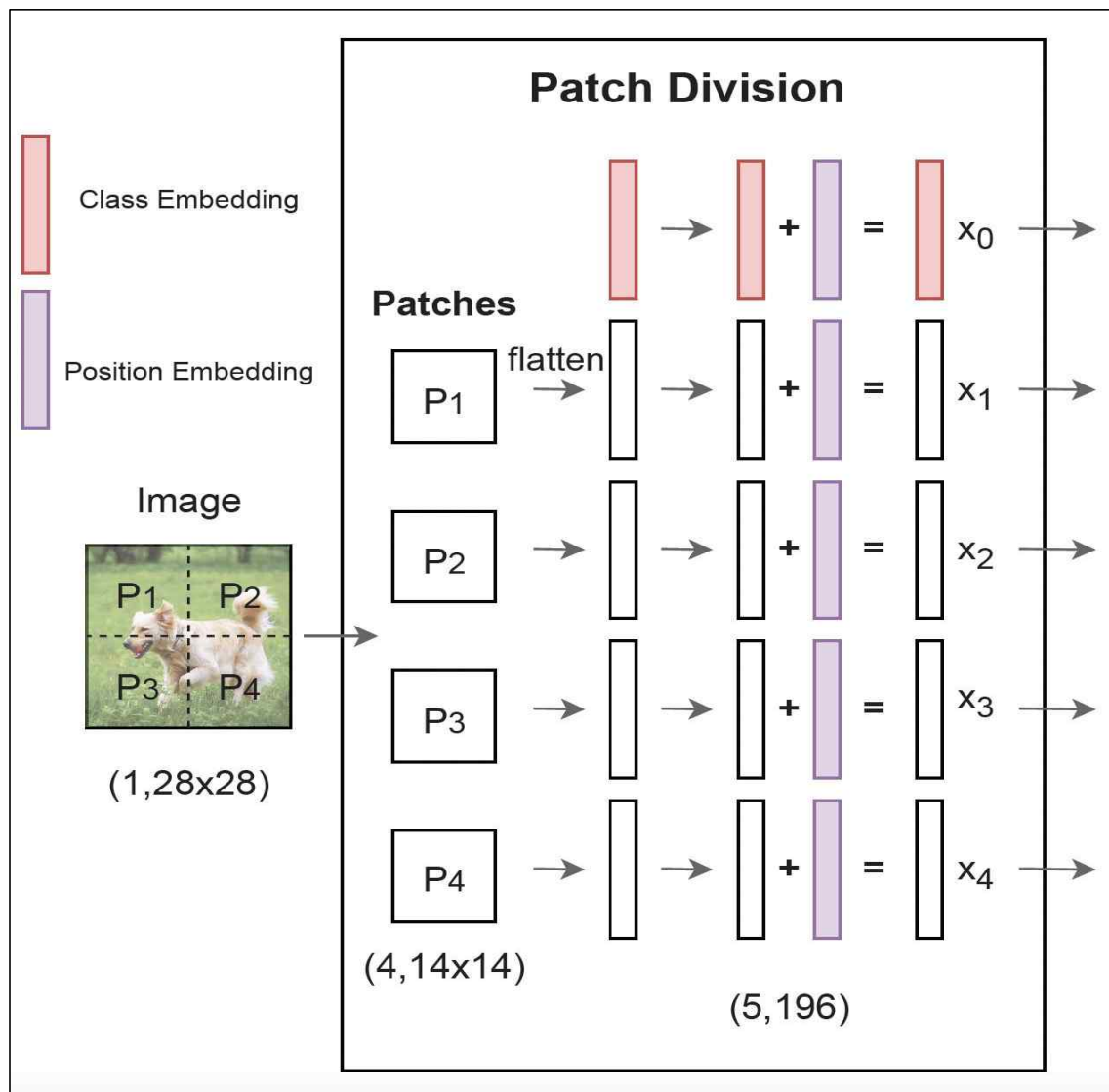


196 Tokens

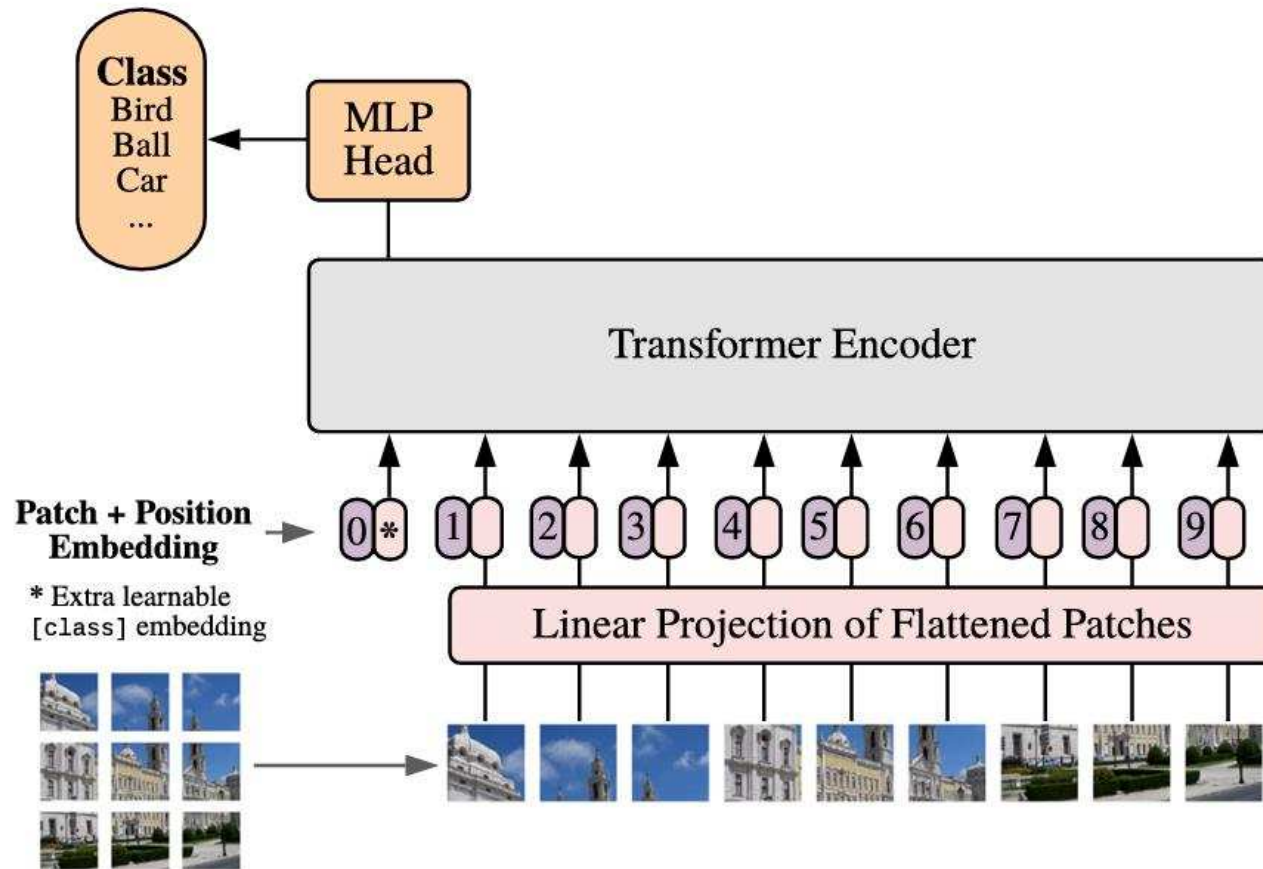
Technical Process

- 1 Split 224×224 image into 16×16 patches
- 2 Flatten each patch (768 values)
- 3 Linear projection to D dimensions
- 4 Add positional encodings
- 5 Result: 196 tokens

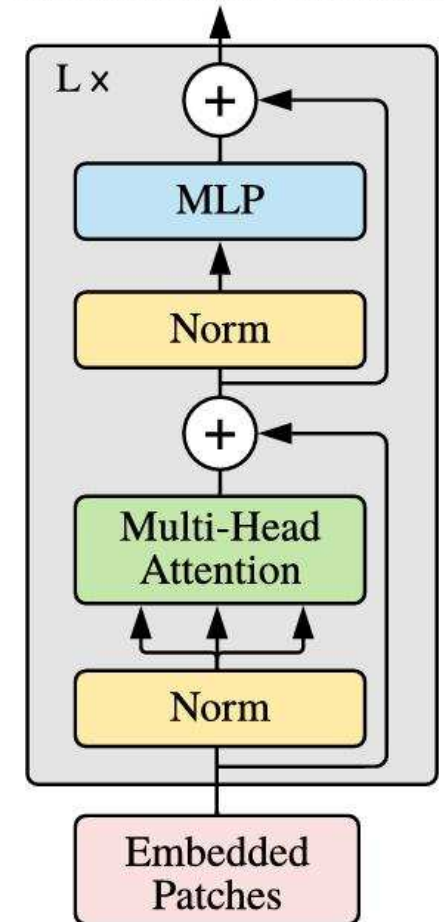
Example: 224x224 → 14x14 grid = 14x14 grid = 196 tokens



Vision Transformer (ViT)



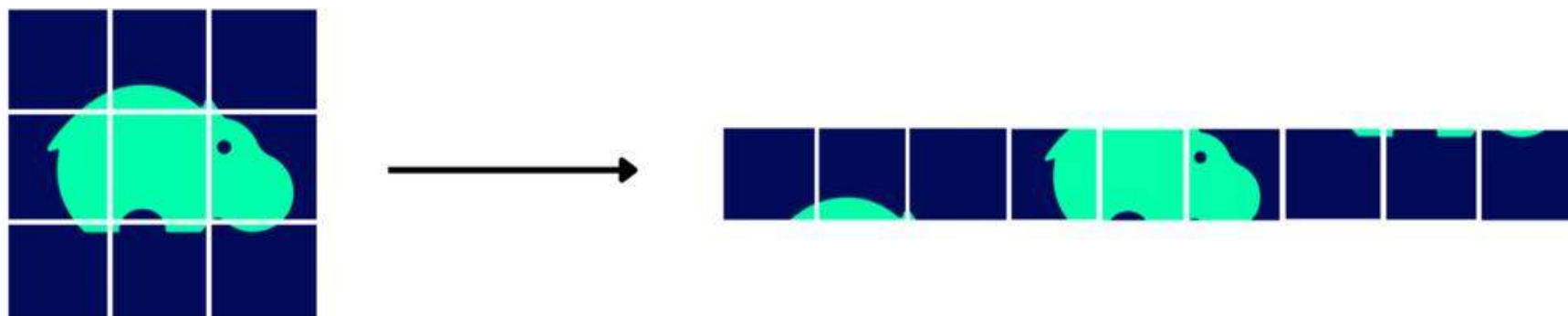
Transformer Encoder



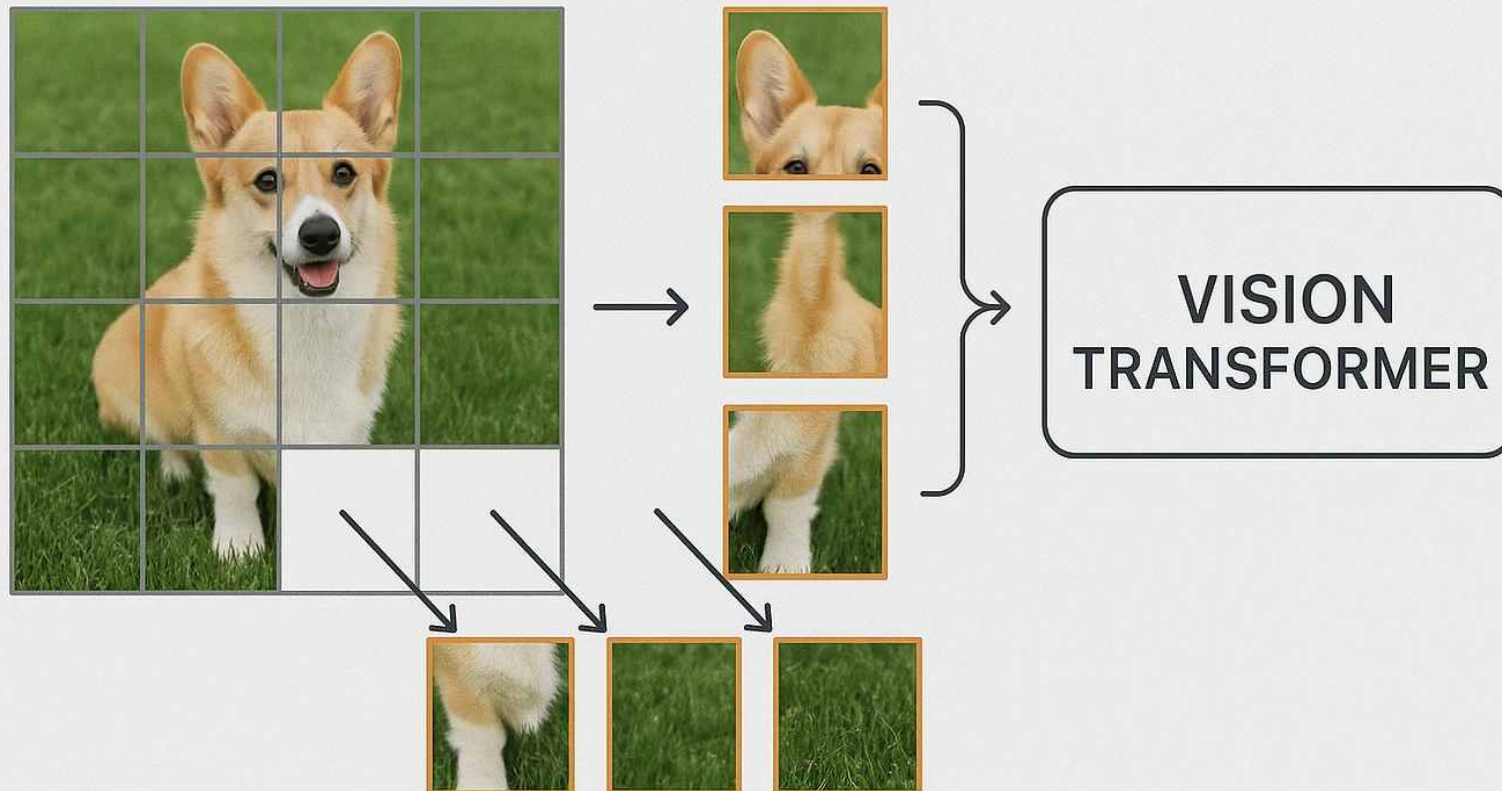
Sentence to word tokens:



Image to image patches:



VISION TRANSFORMER



2. Flatten

각 패치를 1차원 벡터로 펼친다.

예:

$$\begin{aligned} 16 \times 16 \times 3 \\ = 768 \end{aligned}$$

즉 하나의 패치가 768차원 벡터가 된다.

3. Linear Embedding

패치 벡터를 Dense Layer로 변환한다.

즉:

$$768 \rightarrow D$$

보통:

- 768
- 1024
- 1280

차원 등을 사용한다.



4. Positional Encoding

Transformer는 순서 정보를 모르기 때문에 위치 정보를 추가해야 한다.

즉:

- 왼쪽 위 패치인지
- 가운데 패치인지
- 오른쪽 아래인지

알려줘야 한다.

따라서 위치 벡터를 더한다.



5. Self-Attention

ViT의 핵심이다.

각 패치가 다른 모든 패치를 참고한다.

예:

- 고양이 귀 패치
- 눈 패치
- 꼬리 패치

사이의 관계를 학습한다.

Attention은 다음과 같이 계산된다.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

여기서:

- Q = Query
- K = Key
- V = Value

이다.

CNN과 가장 큰 차이

CNN

지역 특징(Local Feature) 중심

Convolution 사용

필터 기반

위치 정보 내장

적은 데이터에서도 강함

계산량 상대적으로 적음

ViT

전체 관계(Global Relation) 학습

Attention 사용

토큰 기반

Positional Encoding 필요

대규모 데이터 필요

Attention 계산량 큼

CNN은 왜 지역 특징에 강한가?

CNN은 작은 커널을 사용한다.

예:

3×3

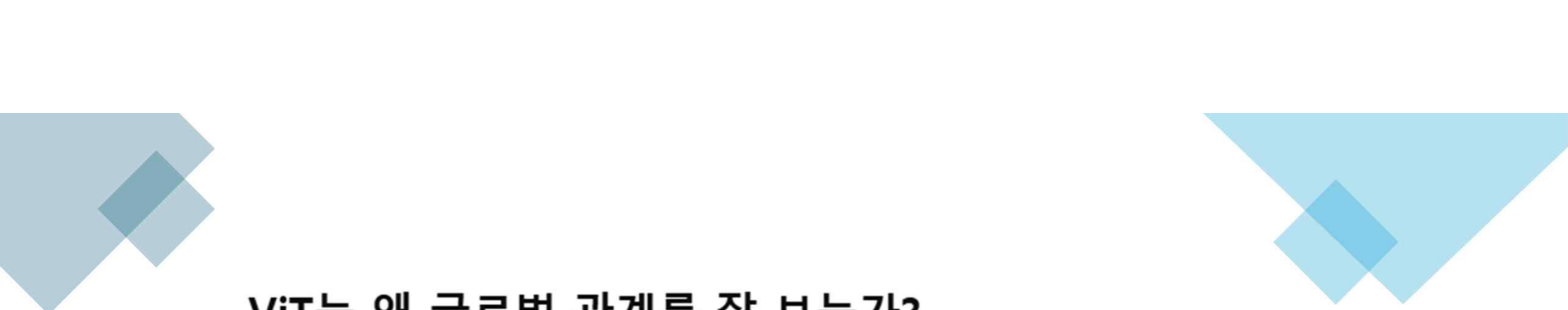
5×5

이런 작은 영역만 본다.

따라서:

- Edge
- Texture
- Pattern

추출에 강하다.



ViT는 왜 글로벌 관계를 잘 보는가?

Self-Attention은 모든 패치끼리 관계를 계산한다.

즉:

- 멀리 떨어진 영역도 연결 가능
- 이미지 전체 문맥(Context)을 이해 가능

예:

- 얼굴 + 몸통 + 꼬리
→ 전체적으로 "고양이"

판단 가능





Self-Attention 계산량 문제

Attention은 패치 수가 많아질수록 계산량이 급증한다.

공식:

$$O(N^2)$$

여기서:

- N = 패치 개수

이다.

예:

- 196개 패치 $\rightarrow$ 196^2 계산

따라서 고해상도 이미지에서는 매우 무겁다.





ViT의 장점

1. 글로벌 문맥 이해

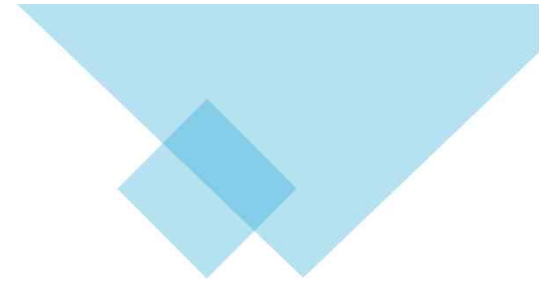
이미지 전체 관계를 잘 본다.

2. 대규모 데이터에서 성능 우수

대형 데이터셋에서 CNN보다 높은 성능 가능

대표:

- ImageNet
- JFT-300M



3. 구조 단순화

CNN처럼:

- 복잡한 Conv 설계
- Pooling 설계

가 필요 없다.





ViT의 단점

1. 데이터 많이 필요

CNN보다 데이터 의존성이 크다.

작은 데이터셋에서는 CNN이 더 좋을 수 있다.

2. 계산량 큼

Attention 비용이 매우 크다.

특히 고해상도 이미지에서 부담이 크다.

3. 학습 시간이 길다

GPU/VRAM 많이 사용





대표 ViT 계열 모델

모델

특징

ViT

기본 구조

DeiT

데이터 효율 개선

Swin Transformer

Window Attention 사용

BEiT

BERT 스타일 사전학습

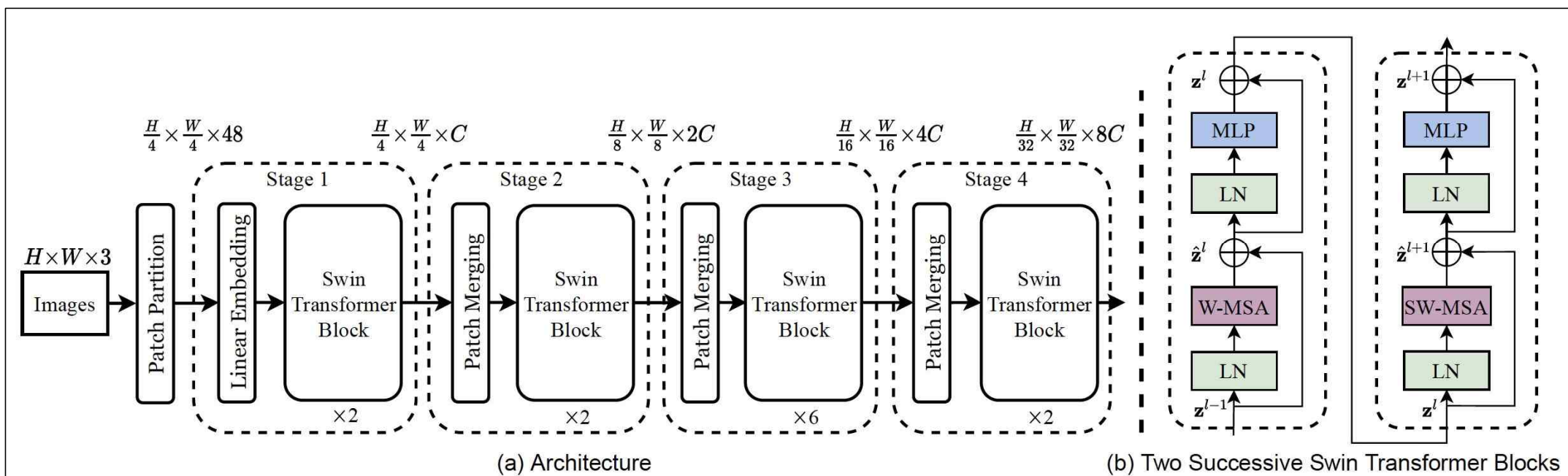
DINOv2

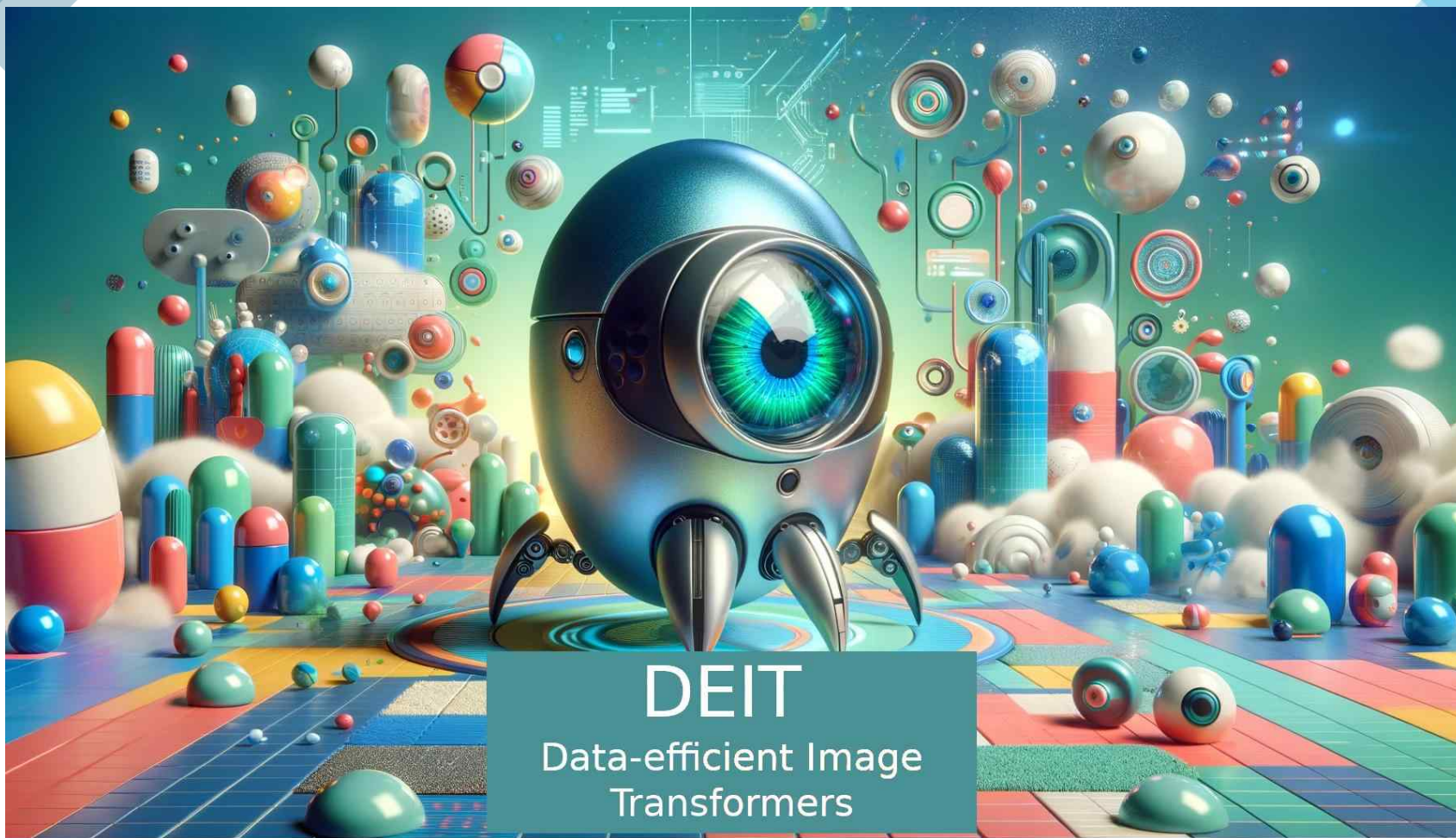
Self-supervised 학습

CLIP

이미지-텍스트 공동학습



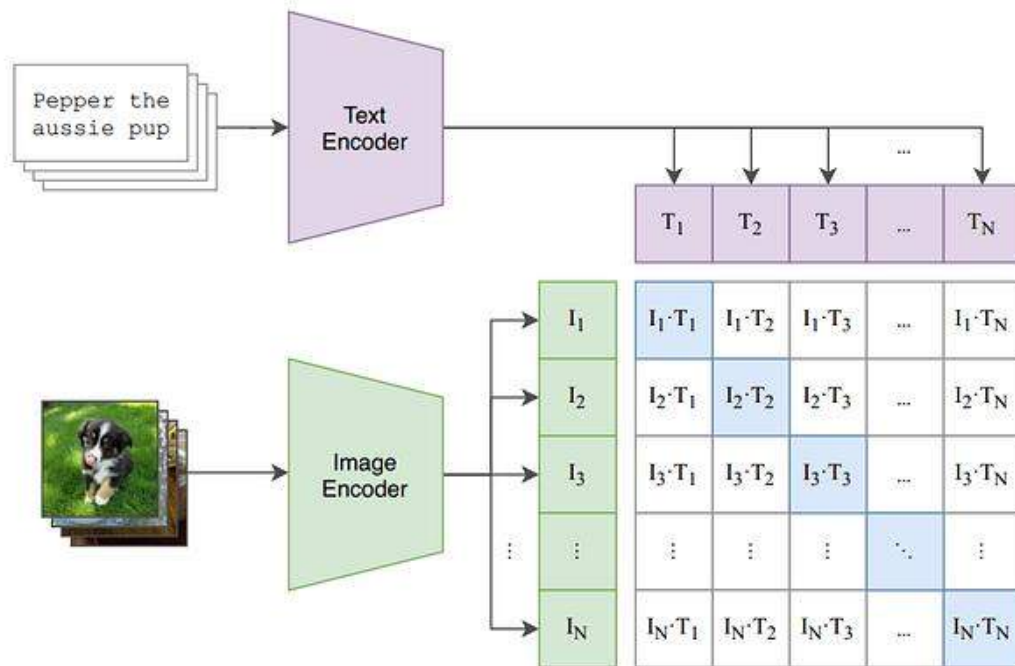




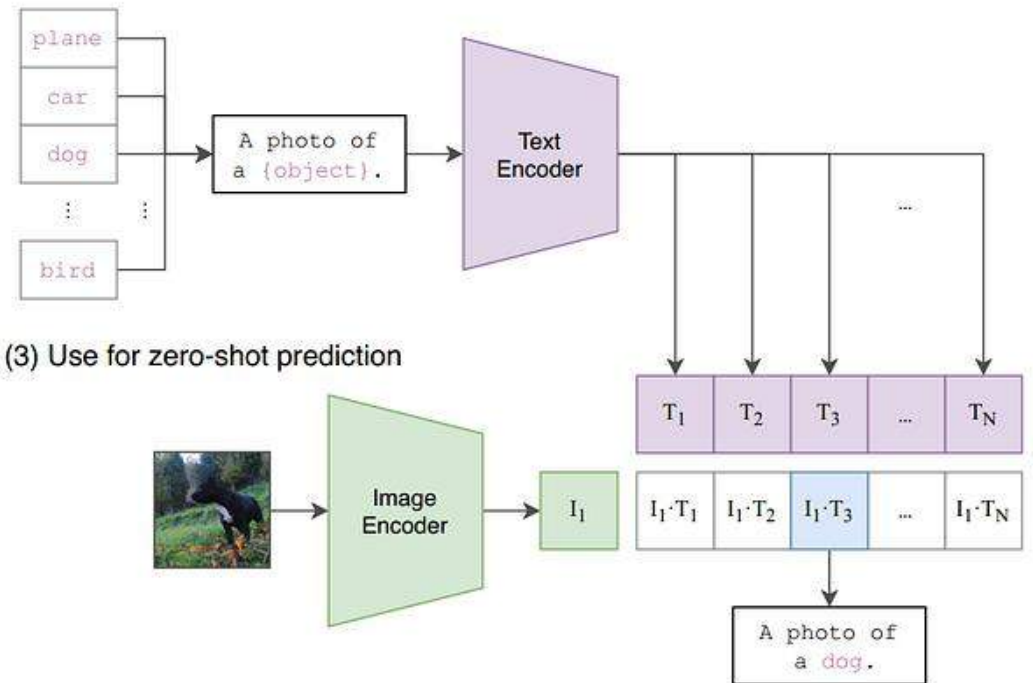
DEIT

Data-efficient Image
Transformers

(1) Contrastive pre-training

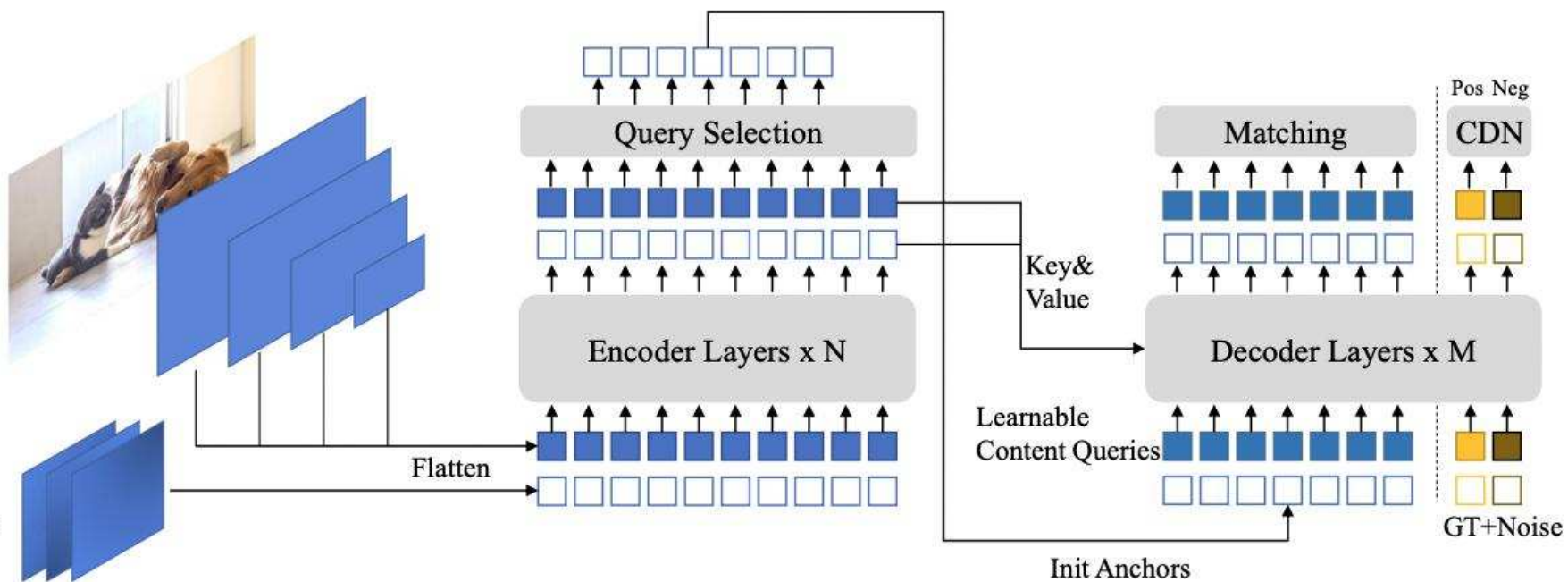


(2) Create dataset classifier from label text



Multi-Scale
Features

Positional
Embeddings



Swin Transformer는 왜 중요한가?

기존 ViT의 문제:

- Attention 계산량이 너무 큼

Swin Transformer는:

- Window 단위 Attention
- Shifted Window

를 사용하여 계산량을 줄였다.

따라서:

- 객체 탐지
- 세그멘테이션

등에서도 매우 많이 사용된다.

대표 모델:

Swin Transformer

CNN vs ViT 실제 경향

최근에는 완전 ViT만 사용하는 경우보다:

- CNN + Transformer 혼합
- Hierarchical Transformer
- Hybrid Backbone

구조가 많다.

실제 산업에서는:

- 속도
- VRAM
- 데이터 크기

문제 때문에 CNN도 여전히 매우 중요하다.

간단한 PyTorch ViT 예제

Python

```
import torch
import timm
from PIL import Image
from torchvision import transforms
```

사전학습 ViT 모델

```
model = timm.create_model(
    'vit_base_patch16_224',
    pretrained=True
)
```

```
model.eval()
```

이미지 전처리

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```

```
img = Image.open("cat.jpg").convert("RGB")
x = transform(img).unsqueeze(0)
```

추론

```
with torch.no_grad():
    y = model(x)
```

```
print(y.shape)
```

Patch Embedding을 직접 구현하면?

Python

```
import torch
import torch.nn as nn

x = torch.randn(1, 3, 224, 224)

patch_embed = nn.Conv2d(
    in_channels=3,
    out_channels=768,
    kernel_size=16,
    stride=16
)

y = patch_embed(x)

print(y.shape)
```

출력:

```
torch.Size([1, 768, 14, 14])
```

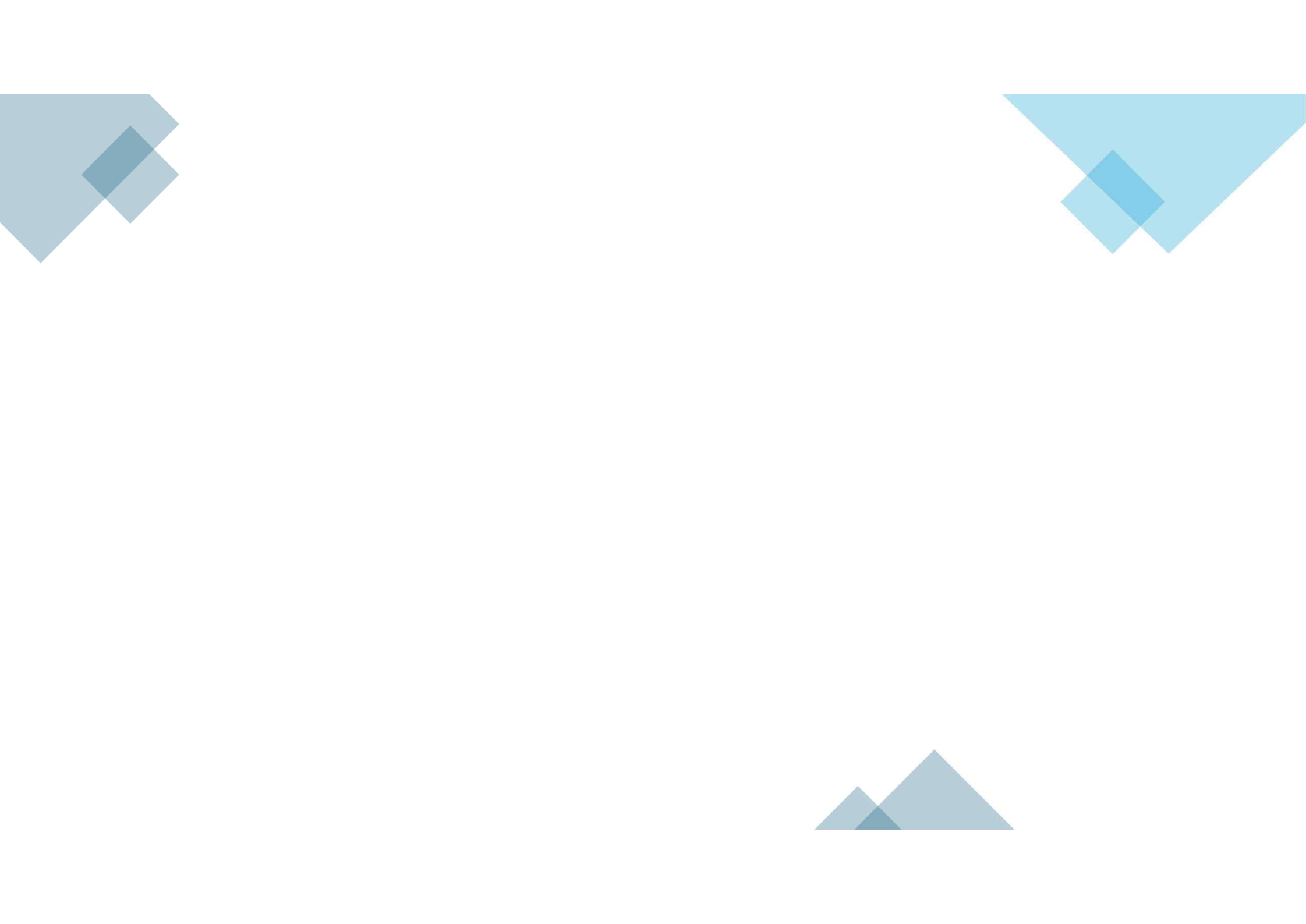
즉:

- 이미지가 패치 단위 특징으로 변환된 것이다.

정리

항목	내용
핵심 아이디어	이미지를 패치 토큰으로 변환
핵심 기술	Self-Attention
장점	글로벌 관계 학습
단점	계산량 큼
CNN과 차이	Convolution 대신 Attention
대표 모델	ViT, Swin, DeiT
많이 쓰이는 분야	이미지 분류, 검출, 세그멘테이션, VLM

Vision Transformer는 이미지를 "단어들의 집합"처럼 처리하여, 패치 간 관계를 Attention으로 학습하는 이미지 Transformer 모델이다.





감사합니다